基礎知識

入力　出力

```
// 入力
string text = Console.ReadLine();

// 出力
Console.WriteLine(text);
```

ちなみに
Console.WriteLine();は１行出力で必ず最後に改行がつきます。
Console.Wirte();で改行無しの出力ができます。

文字列→intの変換

C#の入力受取は基本的にstring型で行います。
そのため、入力を受け取ったら数値に変換する必要があります。

int number = int.Parse(文字列型の数値)　で変換する事が出来ます。

```
// １行取得
string text = Console.ReadLine();

// int型に変換
int number = int.Parse(text);

// int型を出力
Console.WriteLine(number);
```

スペース区切りの文字列を分割

Paizaの入力で数値が複数与えられる時などに
スペース区切りで値が渡される事が多いです。
その際に　string.Split()　を使うことで分割出来ます。

```
// １行取得
string line = Console.ReadLine();

// スペース区切りで分割
string[] texts = line.Split(" ");//""の間に半角スペース
```

スペース区切りの文字列を分割しつつ数値にする

先ほどは文字列を分解していましたが、
それをそのまま数値にすることも出来ます。

```
// １行取得
string line = Console.ReadLine();

// スペース区切りで分割しつつstring →  int
int[] numbers = line.Split(" ").Select(int.Parse).ToArray();
```

２次元配列の受取

```
// ２次元配列の幅を定義（Paizaだと読み取り）
int xLength = 5;
int yLength = 5;

// ２次元配列の変数定義
int[,] matrix = new int[yLength, xLength];

//  中身の読み取り（コピペでOk）
for(int y = 0; y < yLength; y ++)
{
    string s = Console.ReadLine();
    int[] array = s.Split(" ").Select(int.Parse).ToArray();
    for(int x = 0; x < xLength;x++)
    {
        matrix[y,x] = array[x];
    }
}
```

パイザのB問題でよく２次元配列の問題が出てきます。
いちいち書くのも面倒くさいのでコピペで出せるようにしておくと楽です。

備忘録：変なミスをしないために

このプログラムを見て気が利かないと思って欲しい。

```
// １行目の受取
string line = Console.ReadLine();
int[] numbers = line.Split(" ").Select(int.Parse).ToArray();

// ２次元配列の変数定義
int[,] matrix = new int[numbers[1], numbers[0]];

//  中身の読み取り（コピペでOk）
for(int i = 0; i < numbers[1]; i ++)
{
    string s = Console.ReadLine();
    int[] array = s.Split(" ").Select(int.Parse).ToArray();
    for(int j = 0; j < numbers[0]; j++)
    {
        matrix[i,j] = array[j];
    }
}
```

備忘録：変なミスをしないために①

２次元配列でfor文を使う際にはi,jではなくx,yにしましょう。

```
// ２次元配列の幅を定義（Paizaだと読み取り）
int xLength = 5;
int yLength = 5;

// ２次元配列の変数定義
int[,] matrix = new int[yLength, xLength];

//  中身の読み取り（コピペでOk）
for(int y = 0; y < yLength; y ++)
{
    string s = Console.ReadLine();
    int[] array = s.Split(" ").Select(int.Parse).ToArray();
    for(int x = 0; x < xLength;x++)
    {
        matrix[y,x] = array[x];
    }
}
```

```
// ２次元配列の幅を定義（Paizaだと読み取り）
int xLength = 5;
int yLength = 5;

// ２次元配列の変数定義
int[,] matrix = new int[yLength, xLength];

//  中身の読み取り（コピペでOk）
for(int i = 0; i < yLength; i ++)
{
    string s = Console.ReadLine();
    int[] array = s.Split(" ").Select(int.Parse).ToArray();
    for(int j = 0; j < xLength;j++)
    {
        matrix[i,j] = array[j];
    }
}
```

備忘録：変なミスをしないために②

説明用変数を作りましょう。

```
string line = Console.ReadLine();
int[] numbers = line.Split(" ").Select(int.Parse).ToArray();

int xLength = numbers[0];        // 入力1個目はXの個数
int yLength = numbers[1];        // 入力2個目はYの個数
int sarchNumber = numbers[2];    // 入力3個目は検査対象

// ２次元配列の変数定義
int[,] matrix = new int[yLength, xLength];

//  中身の読み取り（コピペでOk）
for(int y = 0; y < yLength; y ++)
{
    string s = Console.ReadLine();
    int[] array = s.Split(" ").Select(int.Parse).ToArray();
    for(int x = 0; x < xLength;x++)
    {
        matrix[y,x] = array[x];
    }
}
```

```
string line = Console.ReadLine();
int[] numbers = line.Split(" ").Select(int.Parse).ToArray();

// ２次元配列の変数定義
int[,] matrix = new int[numbers[1], numbers[0]];

//  中身の読み取り（コピペでOk）
for(int i = 0; i < numbers[1]; i ++)
{
    string s = Console.ReadLine();
    int[] array = s.Split(" ").Select(int.Parse).ToArray();
    for(int j = 0; j < numbers[0]; j++)
    {
        matrix[i,j] = array[j];
    }
}
```

問題を解こう

- 標準入出力　C#編
- シミュレーション　C#編
- 検索履歴　C#編
- 単語のカウント　C#編
- 宝くじ　C#編

プログラミングで稀に使うパターン　（検索履歴）

for文を逆から回すと便利な場合があります。

```
// 4　→　0になるようにfor文を回す
int length = 5;
for(int i = length - 1; i >= 0; i--)
{

}
```

```
apple
book
information
note
pen
pineapple
```

検索履歴の一番新しい情報が一番したにあるので
今回は配列を一番最後の情報から見たほうが
処理しやすい。

プログラミングで稀に使うパターン　（単語のカウント）

配列やリストには便利な関数がたくさんあります。
今回はCountを使うと便利です。

```
// テキストの一覧配列
string[] texts = { "apple", "banana", "apple", "orange", "apple" };

// appleの数をカウントする
int count = texts.Count(x => x == "apple");
```

※　引数にあまり見ない形のやつがいると思いますが後日解説します。

プログラミングで稀に使うパターン　（宝くじ）

下◯桁みたいな時は%演算を使うこといいことが多いです。

```
int baseNumber = 123456;
int number10 = baseNumber % 10;// 6
int number100 = baseNumber % 100; // 56
int number1000 = baseNumber % 1000; // 456
```

プログラミングで稀に使うパターン　（宝くじ）

他にも%は下記の用途で使ったりします。

数値の循環

```
int i = 0;
while (true)
{
    // iが　0　1　2をループする形で循環する
    i = (i + 1) % 3;
}
```

割り切れる場合

```
int angle = 15;

if( angle % 5 == 0)
{
    // 5で割り切れる時だけ通る
}
```

角度の強制

```
int angle = 770;

// 0 ~ 360の値に強制する
angle = ((angle % 360) + 360) % 360;
```